



Documentación Sudoku

Sofía Jael Rivera Vanegas

TEC - Tecnológico de Costa Rica

Escuela de Computación

Taller de Programación - Programa 2: Sudoku

William Mata Rodriguez

10 de junio 2026

Contenido

Enunciado del Proyecto.....	3
Objetivos.....	3
Requerimientos.....	4
Temas Investigados.....	9
Archivos de datos.....	9
Programación dirigida por eventos.....	11
Interfaz gráfica de usuario (GUI).....	13
Desarrollo de GUI con tkinter.....	14
Definición del estilo universal.....	23
Imagen de fondo.....	23
Botón global.....	24
Obtener estado de widget.....	24
Uso de RadioButton.....	25
Capturar el click en la equis de una ventana.....	26
Ayuda con formatear segundos.....	26
Algoritmo de ordenamiento.....	27
Error en pausar reloj.....	27
Obtener el atributo de un widget.....	28
Obtener datetime actual, formato estándar.....	28
Estructura de los archivos usados.....	30
Algoritmo de selección aleatoria.....	30
Archivo "sudoku2026juegoactual".....	31
Archivo "sudoku2026configuración".....	32
Conclusiones.....	33
Problemas Encontrados.....	33
Aprendizajes Obtenidos.....	34
¿Qué haría diferente en un próximo proyecto?.....	34
Estadística de tiempos.....	36
Lista de Revisión del Proyecto.....	37
Referencias bibliográficas.....	38

Enunciado del Proyecto

Sudoku es un pasatiempo bastante popular. Consiste en llenar con dígitos del 1 al 9 cada una de las casillas de una cuadrícula que tiene un total de 9 x 9 casillas. La cuadrícula se puede ver como una matriz de 9 filas y 9 columnas. A la vez esta cuadrícula se divide en subcuadrículas de 3 x 3 casillas.

Para empezar a jugar hay algunos dígitos fijos en la cuadrícula, lo cual determina el nivel de dificultad del juego. Aunque usualmente se usan dígitos, lo que interesa es que sean grupos de 9 elementos diferentes: por ejemplo 9 letras, 9 colores, 9 frutas, etc.

La regla de este juego es que un mismo elemento no se puede repetir:

- En una misma fila (casillas horizontales)
- En una misma columna (casillas verticales)
- En las subcuadrículas (3×3)

Además, los elementos que aparecen al inicio del juego quedan fijos, no se pueden cambiar.

Objetivos

1. Desarrollar habilidades de programación, donde los algoritmos necesarios en este trabajo deben ser de la autoría del estudiante.
2. Adicionalmente se pueden usar herramientas de IA para complementar el trabajo, siempre y cuando su uso sea consciente, crítico y ético. En este proyecto algunos puntos de investigación se pueden llevar a cabo mediante la IA.
3. Aplicar y reforzar aspectos del lenguaje Python 3.

4. Uso de programación estructurada (estructuras de control de ejecución del programa: secuencia, condicional, ciclo, uso de funciones).
5. Aplicar buenas prácticas de programación: documentación interna y externa del programa, nombres significativos, desarrollo y reutilización de funciones, seguir un estilo de programación.
6. Uso de TDA (Tipo de Dato Abstracto): pilas (implementadas con collections.deque)
7. Validar los datos de entrada.
8. Usar archivos de datos.
9. Fomentar algunas habilidades investigativas (lectura-escritura, procesos cognitivos, socialización del conocimiento, obtención de información, análisis, conciencia del autoaprendizaje).

Requerimientos

El programa tendrá una interfaz gráfica de usuario (GUI). Se usarán botones (Buttons widget) para acceder a las funcionalidades del programa, es decir, lo que el programa hace.

A) Jugar

Esta opción permite jugar el Sudoku. El programa tiene una serie de partidas que previamente han sido registradas y de ahí se selecciona aleatoriamente una partida según el nivel de dificultad configurado. Se detallan las diferentes funcionalidades (botones).

Iniciar Juego:

- Esté botón inicia el juego.
- El programa permite seleccionar del panel de elementos uno de ellos haciendo clic sobre ese elemento, e insertarlo en la cuadrícula.

- Se hacen las validaciones necesarias cuando el jugador inserta un elemento a la cuadrícula para que la jugada cumpla con las reglas del juego, de lo contrario se le envía el mensaje correspondiente.
- El programa guarda la partida en el archivo tipo json "sudoku2026_bitácora_jugadas.json".
- Antes de iniciar el juego el jugador debe dar un nombre.
- Luego de dar el botón que inicia el juego esté se deshabilita.
- Se puede modificar el timer en la configuración o antes de empezar. Siempre debe ser mayor a cero alguno de sus valores.

Deshacer jugada:

- Elimina la última jugada que se hizo dejando la casilla vacía.
- Al deshacer una jugada se crea un elemento en la pila de jugadas eliminadas.
- En caso de que no hayan jugadas para borrar según la pila hay que enviar el mensaje NO HAY JUGADAS PARA DESHACER.
- Se puede seleccionar esta opción solamente si el juego ha iniciado.
- La pila de jugadas realizadas se reinicia en cada juego.

Rehacer jugada:

- Rehace la última jugada que se eliminó.
- Al rehacer una jugada se crea un elemento en la pila de jugadas realizadas.
- En caso de que no hayan jugadas para rehacer según la pila hay que enviar el mensaje NO HAY JUGADAS PARA REHACER.
- Se puede seleccionar esta opción solamente si el juego ha iniciado.
- La pila de jugadas eliminadas se reinicia en cada juego

Borrar juego:

- Se devuelve al usuario a la opción de Jugar usando la misma partida pero eliminando todas las jugadas que hizo
- Se puede seleccionar esta opción solamente si el juego ha iniciado

Terminar Juego:

- Termina de inmediato el juego y se vuelve a mostrar otro juego como si estuviera entrando a la opción de Jugar.
- Se puede seleccionar esta opción solamente si el juego ha iniciado.

Top X:

- Detiene el reloj si se está usando.
- Despliega una pantalla con los mejores tiempos en cada nivel.
- El despliegue de las jugadas es en orden ascendente por tiempo.
- El despliegue del TOP X se hace en un archivo tipo PDF.

Guardar juego:

- Guarda en el archivo "sudoku2026juegoactual" el juego actual.
- Contiene la última jugada guardada por cada jugador y nivel de dificultad.

Cargar juego:

- Trae del archivo "sudoku2026juegoactual" el juego que tenga con su configuración y lo pone en la ventana como el juego actual.
- Si en el archivo no hay una partida para el jugador con ese nivel de dificultad hay que enviar el mensaje NO TIENE UN JUEGO GUARDADO CON ESTA DIFICULTAD.

B) Configurar

Esta opción es para indicar las condiciones con que se va a jugar. Contiene los siguientes datos que se van a guardar en "sudoku2026configuración.json". La estructura del archivo: diccionario con las llaves "nivel", "reloj", "top x", "elementos".

Nivel:

- El nivel debe dar las tres opciones de dificultad, fácil, intermedio y difícil.

Reloj:

- El reloj debe dar tres opciones, cronómetro, timer y sin reloj.
- Si se usa el Timer se deben insertar los datos de horas, minutos y segundos.
- Los datos también pueden ser modificados por el jugador en la opción Jugar.
- Las horas pueden estar entre 0 y 4.
- Los minutos entre 0 y 59.
- Los segundos entre 0 y 59.
- El timer debe tener al menos uno de estos valores.

Jugadas del Top X:

- Permite al usuario insertar la cantidad de jugadas desplegadas en el TOP X.
- Por defecto es 0.
- Debe ser un entero entre 0 y 10.
- 0 significa que despliega todas las jugadas que estén en cada nivel.
- Un valor entre 1 y 10 significa que despliega ese máximo de jugadas.

Panel de elementos:

- El usuario elige el panel de elementos para llenar la cuadrícula.
- Los números siempre serán del 1 al 9 y para letras de la A a la I.

C) Ayuda

Esta opción debe desplegar el manual del usuario ,

D) Acerca de

Esta opción despliega información "Acerca del programa" donde están los datos del nombre del programa, la versión, la fecha de creación y el autor.

E) Salir

Esta opción se usa para salir del programa.

Temas Investigados

En esta sección se va a dar una breve explicación de temas investigados, los cuales están relacionados con el proceso para llevar a cabo este proyecto.

Archivos de datos

Una de las formas de los programas para retener información son las variables, estas guardan información en un espacio en la memoria, sin embargo, la información que se guarda en estas es efímera. Es decir, cuando se cierre el programa, el valor almacenado en la variable se pierde. Es por ello que se crearon los archivos de datos

Los archivos de datos son una: "herramienta útil para almacenar diferentes tipos de datos. Pueden ser conjuntos de datos, diagramas detallados o información específica de un programa" (Adobe, 2026). Su importancia radica en que, gracias a estos se puede guardar información importante para el programa, generalmente se guarda en el disco duro de la computadora, es decir en un archivo.

Para generar un archivo, primero se indica su nombre, seguido por un punto y el tipo de archivo que es, es decir, su extensión, esto para brindar mayor información acerca de su contenido, y sus características. Un ejemplo claro es el tipo de "archivo.txt".

Para efectos del programa se investigan cuatro tipos de archivos bastante usados en la programación. Explicados de manera general son los siguientes.

Archivos de tipo JSON, o JavaScript Object Notation, representado por la extensión ".json" es un: "formato de almacenamiento e intercambio de datos que utiliza una estructura de clave-valor para representar información de manera legible tanto para humanos como para máquinas" (García, 2025). Podría decirse que tiene bastante similitud con un diccionario, ya que tenemos una clave, es

decir el id único del elemento seguido de su valor, su diferencia está en que se guardan de manera permanente.

Archivos de tipo CSV, es un acrónimo de, valores separados por comas, su extensión es ".csv", usa un formato de texto plano, lo cual ayuda a simplificar el almacenamiento de los datos y su posterior lectura. Los datos se almacenan en forma de tabla, solamente que las columnas se separan por comas, y cada fila es un registro.

Archivos de texto (.txt), como su nombre lo indica es un archivo que guarda texto, sin embargo, esté no presenta ningún formato, es decir, no requiere de ningún programa independiente para abrirlo o editarlo. Actualmente se ofrece gran variedad de aplicaciones, como "Word", "TextEdit", "Google Docs". Además una característica importante de ese tipo de archivos es que su contenido almacenado y los conjuntos de caracteres se interpretan como una secuencia.

Archivos de tipo binario, son un tipo de archivo cuyo: "contenido está en formato binario y consiste en una serie de bytes secuenciales , cada uno de los cuales tiene una longitud de ocho bits . El contenido debe ser interpretado por un programa" (Sheldon, 2022)..

Uso en el Proyecto

Para efectos de esté proyecto, el tipo de archivo que se implementó fue el de tipo JSON. Como se explicó, a la hora de guardar datos esté tipo de archivo es bastante eficiente.

Se usó en varias partes del programa, primero se importa al principio del programa, con la línea de código **"import json"**, luego se puede usar todas sus funcionalidades.

Para grabar datos en un archivo tipo json primero se debe definir el nombre del archivo que se desea crear, lo siguientes es un ejemplo de la forma en que se implementó para la configuración por default.

```
nombre_archivo_config = "sudoku2026configuración.json"
```

Luego, se crea el objeto o la información que se requiere guardar en el archivo tipo JSON, para esté caso, se requiere grabar un diccionario como el siguiente.

```
configuracion_default = {
    "nivel": "fácil",
    "reloj": ["cronometro",0,0,0],
    "top x": 0,
    "elementos": "numeros"
}
```

Una vez definido el nombre del archivo y la información a guardar solo se necesita abrir en archivo para escribir en él, luego insertar el diccionario creado y guardarlo.

```
f = open(nombre_archivo_config, "w")
json.dump(configuracion_default, f)
f.close()
```

Programación dirigida por eventos

La Programación Orientada a Eventos, por sus siglas POE, se trata de un estilo de programación diferente al secuencial, esté se basa en la interacción de lo externo con el programa, es decir, la ejecución del programa está relacionada con

eventos que no se sabe cuándo ocurrirán. Normalmente debido a la interacción de un usuario, como apretar un botón.

Este estilo de programación en lugar de ser secuencial, permite que: “los diferentes componentes del sistema respondan a eventos que ocurren de manera asíncrona, como la interacción del usuario, la llegada de datos externos o cambios en el estado del sistema” (Martínez, 2025). Para lograr esto, es usual que se definan eventos para las interacciones con el usuario.

Además la POE se usa en gran manera a la hora de desarrollar páginas web o aplicaciones, ya que puede utilizarse en el desarrollo de interfaces de usuario, que sean interactivas.

Uso en el Proyecto

En el proyecto, la mayoría de los widgets poseen un función asociada, es decir, son eventos. Por ejemplo, a la hora de tocar un botón, éste tiene una función específica asociada, e irá a está una vez que el usuario interactúe con el.

Se implementó de forma de programar de POE en los botones, por ejemplo, en el menú principal, cada botón al ser apretado (darle “click”) va a ejecutar una función. El primer botón de menú abre el juego, el segundo es para cambiar la configuración, sigue el de ayuda, que despliega el manual de usuario, luego acerca de, y por último el de salir.

Para un botón, la forma de asociar un evento se muestra a continuación, usando el ejemplo del botón para desplegar el juego. Se define el botón y su estructura.

```

botonA = tk.Button( root,                                #Se define el botón y la ventana
                    text = "Jugar",                        #El texto que tiene el botón
                    command = abrir_ventana_jugar,        #Esta es la función que llamará

```

****ESTILO_BOTON_MENU)**

#El estilo del botón

Interfaz gráfica de usuario (GUI)

La interfaz gráfica de usuario es una manera visual de interactuar con el usuario, es decir, no es necesario aprenderse comandos o entrar al cmd, todo se hace por medio de la pantalla, ya sea por botones, barras, menús, elementos visuales, etc. Uno de los ejemplos más usados es la interfaz de Windows, ya que no es necesario usar comandos para abrir una aplicación determinada, tan solo se da click, o en la barra de búsqueda.

Para la GUI es bastante importante el uso de símbolos o íconos, ya que los elementos guían al usuario sin necesidad de usar palabras, el nombre del elemento puede estar en miles de idiomas, mientras que el ícono es conocido a nivel mundial, un claro ejemplo del icono de wifi.

La GUI es bastante importante, se puede definir como un traductor entre el usuario y el sistema o su máquina, sin la GUI habría que abrir el cmd y ejecutar un comando cada vez que se desee abrir una aplicación.

Uso en el Proyecto

En el caso de este proyecto, la interfaz gráfica es sumamente importante, ya que, es la forma en que el usuario puede comunicarse e interactuar con el juego. Jugar sudoku sin una interfaz gráfica sería demasiado complicado.

Para llevar a cabo esta interfaz, se implementa la herramienta conocida como Tkinter, esta posee widgets y elementos, como botones, títulos, entradas de texto, etc..

Desarrollo de GUI con tkinter

En este apartado se describen los elementos de tkinter que se usaron para desarrollar este programa 2. Lo primero que se abordará serán los **widgets implementados**.

Widget Label: este es uno de los widgets más usados, debido a su facilidad para implementar. Este se trata de un simple fragmento de texto de una sola línea que se puede colocar en la ventana (root), esta puede tener cualquier texto. Para definir un widget tipo label se hace con la siguiente estructura.

```
label = tk.Label(root, text="Hello")
```

Widget Button: este widget también es bastante usado dentro de tkinter, debido a que es una de las formas más interactivas para que el usuario interactúe con el programa. Este widget representa un botón, que al pulsarlo activa una acción. Estas acciones pueden ser: abrir otra ventana, desplegar un mensaje, entre otros. Su estructura es la siguiente.

```
button = tk.Button(root, text="Botón", command=saludar)
```

La acción que hará el botón se define con la palabra reservada command, en este caso lo que hará el botón será saludar.

Widget Entry: este es un cuadro que se puede editar, insertando texto, de una sola línea, cabe recalcar que puede ser modificado por el usuario. Este widget es usado normalmente para introducir datos que se desean recopilar, como un formulario, un nombre, un correo. La forma de crear un entry puede variar según el programador, pero sigue una forma particular. En este caso se siguen varias líneas de código.

```
entry = tk.Entry(root) #Se crea el entry dentro de la ventana (root)
```

```
entry.insert(0, "Enter your text") #Se usa el insert para agregarle un texto
```

```
entry.bind("<Return>", return_pressed) #Es la acción que hará
```

Widget Radio Button: los radio buttons son bastante usados cuando se desea que el usuario seleccione una de las opciones que se despliegan, sin que tenga que escribir, lo que se va a guardar es lo que el usuario marcó.

Para crear la estructura de la RadioButton primero se define una variable de la siguiente manera:

```
selected = tk.StringVar()
```

Esta variable sirve para agrupar los radios, y que de estos solo se pueda seleccionar una. Luego se definen los valores de los radios.

```
r1 = tk.Radiobutton(root, text='Op1', value='1', variable=selected)
```

```
r2 = tk.Radiobutton(root, text='Op2', value='2', variable=selected)
```

```
r3 = tk.Radiobutton(root, text='Op3', value='3', variable=selected)
```

Aparte de los widgets que existen en tkinter también hay **gestores de geometría**, los cuales se encargan de controlar las estructuras, el diseño de la ventana. Actualmente tkinter posee tres gestores de geometría.

Gestor Pack: es el más usado, usa un algoritmo de empaquetado, esto para colocar los widgets en un frame de la ventana en un orden específico, se colocan hacia abajo, uno debajo de otro. Esto lo logra mediante dos pasos.

Se calcula un área rectangular, esta debe tener el tamaño suficiente para contener el widget.

Además se encarga de centrar el widget en el frame a menos que se especifique una ubicación diferente.

```
frame = tk.Frame(root, width=100, height=100)
```

```
frame.pack()
```

Gestor Place: en principio este gestor es similar a pack, debido a que se encarga de colocar los widgets, sin embargo, difiere en que el programador puede definir donde desea colocar el widget que acaba de crear. Se deben proporcionar dos argumentos, que especifican las coordenadas x e y de la esquina superior izquierda del widget.

```
label = tk.Label(root, text="texto")
```

```
label.place(x=75, y=75)
```

Gestor Grid: este gestor también funciona de manera similar que el pack, sin embargo, funciona dividiendo la ventana, o el frame en el que está, en columnas y filas. Para empaquetar el widget se pasa dentro del grid dos parámetros, el número de fila y el número de columna.

```
frame = tk.Frame(root)
```

```
frame.grid(row=1, column=2)
```

Cuando se trabaja con tkinter también se pueden implementar módulos que existen dentro de esta librería, estos deben importarse siempre al inicio del programa. El módulo que se usó en el programa se conoce como **Messagebox**, como su nombre lo indica, es cuadro de diálogo es una ventana con un título, puede contener un mensaje, un ícono y uno o más botones. Es implementado usualmente para informar al usuario de algún error, además, se pueden hacer preguntas y dar datos.

Es normal definir un messagebox dentro de una función, para que se ejecute cuando el programador lo requiera. Su estructura usual es la siguiente.

```
from tkinter import messagebox
```

```
messagebox.showinfo(message="Mensaje", title="Título")
```


En este caso se usó el método **showinfo()**, que se encarga de mostrar la información requerida. Aparte de este método existen muchos más, entre los cuales se encuentran los siguientes.

- showinfo(), desliza información
- showwarning(), para alertar de una situación
- showerror(), para informar de un error
- askyesno(), cuando se requiere hacer una pregunta de si/no

Otra funcionalidad bastante usada en tkinter es la creación de ventanas secundarias. Cabe destacar que la ventana principal se crea a partir de la clase tk.Tk, normalmente se crea al inicio, y controla la aplicación y su ejecución.

Una ventana secundaria se crea en respuesta a un evento, para esto se emplea la clase **tk.Toplevel**. Cuando el usuario cierra la ventana principal, todas las ventanas secundarias asimismo se cierran, finalizando la ejecución del programa. Sin embargo las ventanas secundarias pueden cerrarse y abrirse.

El fragmento de código que sigue define una ventana secundaria con la clase Toplevel.

```
ventana_secundaria = tk.Toplevel()
```

En realidad, su definición es bastante sencilla, sin embargo, debe tenerse cuidado cuando se agreguen widgets y elementos a la ventana secundaria, ya que al definir el widget debe especificarse en qué ventana se está creando.

```
label = tk.Label(ventana_secundaria, text="Hola" )
```

Uso en el Proyecto

Para mayor claridad, este apartado explica y ejemplifica el uso que se le dió a cada elemento mencionado anteriormente, por temas de comodidad, se irán abarcando según el orden del apartado anterior.

Widget Label: este fue uno de los widgets más usados, ya que es una forma relativamente sencilla de mostrar información al usuario. Se implementó en varias partes, como en el menú principal, en la ventana de juego, en la configuración, y el acerca de, se usó principalmente para la creación de los títulos de cada sección, y alguna información necesaria.

```
label = tk.Label(root,                #La ventana de donde pertenece

    text="Menú Principal",           #El texto que tendrá

    font=('Segoe UI', 30, "bold"),    #El tipo de letra y tamaño

    compound="left",                 #La posición que debe tener

    fg="#0c3a95"                     #El color de la letra

).pack()                            #Se crea y guarda dentro de la ventana
```

Widget Button: los botones son un elemento esencial en el programa, ya que son la forma más eficaz de interactuar con el juego. Los botones están en casi todo el juego, en el menú principal se usan botones para las opciones, en la ventana de jugar, se usan para seleccionar los elementos y las opciones del juego.

```
boton_top_x = tk.Button(frame_botones_jugada2,

    font = TEXTO_UNIVERSAL,

    text = f"Top X",

    bg = "#ffdddd",

    activebackground = "#eeeab1",

    **ESTILO_BOTONES,

    command = mostrar_topx )
```

Widget Entry: los entry se usaron bastante para obtener información que el usuario debe digitar. Por ejemplo, su nombre de jugador, la cantidad de jugadores en el Top X, además de la información que requiere el timer. Además, se usó *widget entry* en cada celda de la matriz, para poder insertar números y cambiarlos con comodidad.

```
horas = tk.Entry(frame_b_aux,width=3, justify="center", relief=tk.GROOVE)
```

```
horas.bind("<FocusOut>", valida_horas) #Llama a la función si pierde el foco
```

```
horas.bind("<Return>", valida_horas) #Llama a la función si da enter
```

Widget Radio Button: el único lugar donde se usó este widget fue en la configuración, ya que el usuario debe seleccionar el nivel (fácil, intermedio, difícil), el tipo de reloj (cronómetro, timer, sin reloj) y el panel de elementos que se usará (letras, números).

El siguiente es un ejemplo del radio en el tipo de reloj.

```
root_config.reloj_seleccionado = tk.StringVar(value="cronometro")
```

```
radio_cron = tk.Radiobutton(frame_b,
    text="Cronómetro",
    variable=root_config.reloj_seleccionado,
    value='cronometro',
    **ESTILO OPCIONES_CONFIG,
    command=muestra_timer)
```

```
radio_cron.pack(pady=10, padx=5)
```

Y así se sigue con las demás opciones, si se requiere agregar más solo se usa la misma variable.

Para los **gestores de geometría**, cabe resaltar que se hizo uso de todos los que se mencionaron, aunque el menos usado fue el `place()`.

Gestor Pack: este gestor se implementó para aquellos elementos que no requerían un posicionamiento específico en la ventana. Un ejemplo de esto es en el menú principal, aquí cada botón y label está dentro de la ventana usando un `pack()`. Se muestra el caso del título del menú.

```
label = tk.Label(root,                #La ventana de donde pertenece
                 text="Menú Principal",  #El texto que tendrá
                 font=('Segoe UI', 30, "bold"), #El tipo de letra y tamaño
                 compound="left",        #La posición que debe tener
                 fg="#0c3a95"           #El color de la letra
                 )

label.pack()
```

Este ejemplo es el botón de salir, todos los botones están en cascada.

```
botonE = tk.Button(root,
                   text="Salir",
                   command=quit,
                   **ESTILO_BOTON_MENU )

botonE.pack(pady=15)
```

Gestor Place: aunque su uso en el programa es muy poco, se puede ver su funcionamiento. El único momento donde se implementó este gestor fue en el menú principal, específicamente para colocar la imagen de fondo.

```
bg = tk.PhotoImage(file = "img/sudoku-fondo.png")
```

```
label_fondo = tk.Label(root, image = bg)
```

```
label_fondo.place(x = 0, y = 0, relwidth=1.0, relheight=1.0)
```

Gestor Grid: este gestor fue el más usado en la ventana del juego, ya que habían muchos elementos y botones que debían estar acomodados en filas, columnas etc... Sin embargo, no solo se usó grid, también pack() para acomodar los elementos de grid aparte.

Es importante mencionar lo siguiente. Cuando se trabaja con tkinter, la forma de acomodar los elementos es importante, ya que los gestores no pueden combinarse entre sí, es decir, si hay varios elementos en un mismo nivel, estos deben usar un solo gestor para estar acomodados.

Para ejemplificar de la mejor manera, un ejemplo. La ventana puede tener muchos widgets, estos deben estar organizados con un solo gestor, sin embargo, dentro de un widget, digamos un Frame, pueden haber más elementos, y su vez, estos pueden tener otro gestor, pero entre ellos debe haber el mismo.

```
horas.grid(row=2, column=0, sticky=tk.NSEW, ipady=5)
```

```
minutos.grid(row=2, column=1, sticky=tk.NSEW, ipady=5)
```

```
segundos.grid(row=2, column=2, sticky=tk.NSEW, ipady=5)
```

Note que cada widget que usa grid está en una columna o fila diferente a los demás. En este caso, todos están en la misma fila pero alineados hacia la derecha.

En el caso del módulo **MessageBox**, se le dió un uso bastante repetitivo, ya que hay muchas situaciones donde es necesario alertar al usuario de una situación. Entre estos, indicarle un error al insertar un elemento, a la hora de

terminar o borrar el juego, si el usuario no ha digitado su nombre y le da click al botón de iniciar juego, etc...

```
msg = "Esta casilla ya tiene un elemento fijo"
```

```
messagebox.showinfo(message=msg, title="Error")
```

Por último, para cada funcionalidad del menú principal (cada botón) se usó la clase **tk.Toplevel**, para crear las ventanas secundarias, a excepción del botón salir y ayuda.

```
root_acerca = tk.Toplevel()
```

```
root_acerca.title("Acerca del Sudoku")
```

```
root_acerca.geometry("700×700")
```

Se destaca lo siguiente. Es sumamente importante tener en cuenta el nombre que se le da a la ventana secundaria, ya que si se desea tener un elemento en dicha ventana, debe usarse ese nombre y no el de la ventana principal, sino el elemento quedaría mal formado o dando errores.

```
frame_acerca = tk.Frame(root_acerca, padx= 100, pady=50)
```

Esté es un elemento creado para estar dentro de la ventana secundaria de "root_acerca".

Definición del estilo universal

Se hizo uso de la IA para encontrar una forma de definir estilos universales para no tener que hacerlo cada vez que se crea un nuevo widget o elemento de la ventana.

Objetivo del uso	Creación de un estilo universal
Herramienta utilizada	Gemini
Prompt o pregunta	Usando Tkinter, enséñame una forma de crear un estilo universal para los botones
Respuesta	<pre> FUENTE_UNIVERSAL = ("Segoe UI", 15, "bold") ESTILO_BOTON_MENU = { "font": FUENTE_UNIVERSAL, "bg": "#2e3440", # Gris oscuro nórdico } btn = tk.Button(contenedor, text="Botón", **ESTILO_BOTON_MENU) </pre>
¿Cómo usó o adaptó la respuesta?	Realmente no use todo el código que me dió la IA, ya que no lo necesitaba realmente, solo ocupaba la idea. Tome el concepto de poder usar un estilo dentro de otro estilo.
Reflexión crítica	La IA me ayudó bastante a simplificar la tarea de ir definiendo un mismo estilo una y otra vez en cada botón o texto que tenga.

Imagen de fondo

Se hizo uso de la IA para ponerle un fondo al juego de sudoku

Objetivo del uso	Lograr establecer un fondo para la ventana
Herramienta utilizada	Gemini
Prompt o pregunta	¿Cómo puedo poner una imagen en el fondo de la ventana?

Respuesta	<pre># 1. CARGAR LA IMAGEN CON PHOTOIMAGE imagen_fondo = tk.PhotoImage(file="tu_imagen.png") # 2. CREAR EL LABEL DEL FONDO # Usamos .place() para clavarlo en la esquina superior izquierda (0,0) lbl_fondo = tk.Label(ventana, image=imagen_fondo) lbl_fondo.place(x=0, y=0, relwidth=1.0, relheight=1.0)</pre>
¿Cómo usó o adaptó la respuesta?	No le hice mucho cambio, solo puse los nombres de mis variables y listo, no tenía errores entonces no hubo necesidad de cambiar el código.
Reflexión crítica	En realidad el código que me dió la IA me sirvió de maravilla

Botón global

Se hizo uso de la IA para obtener el valor de un botón global

Objetivo del uso	Obtener el valor de un botón global
Herramienta utilizada	Gemini
Prompt o pregunta	Obtener el valor de un botón si tengo dicho botón como variable global
Respuesta	<code>texto_extraido = boton_global.cget("text")</code>
¿Cómo usó o adaptó la respuesta?	Se usó para varios botones que están en el programa.
Reflexión crítica	Me ayudó bastante, logré resolver el problema

Obtener estado de widget

Se hizo uso de la IA entender mejor cómo se usa cget()

Objetivo del uso	Obtener el estado de un widget cualquiera con <code>cget("state")</code>
------------------	--

Herramienta utilizada	Gemini
Prompt o pregunta	¿Cómo obtener el estado de un widget?
Respuesta	<pre># OBTENER EL ESTADO: # Usando .cget() estado_actual = entrada.cget("state") # Alternativa corta (estilo diccionario): estado_actual = entrada["state"]</pre>
¿Cómo usó o adaptó la respuesta?	El que preferí usar fue el primero que la IA me dió, el segundo no lo use, pero sirvió mucho, se usa a lo largo del programa para obtener los estados de varios entry.
Reflexión crítica	La verdad me ayudó bastante, usé lo que me dio y sirvió.

Uso de RadioButton

Se hizo uso de la IA para poder saber más acerca de los radios buttons.

Objetivo del uso	Saber como seleccionar solo un radio al inicio del programa
Herramienta utilizada	Gemini
Prompt o pregunta	Usando radio button en tkinter, ¿como hago que no todo los radios se seleccionen de una vez?
Respuesta	<pre># Valor por defecto para que no inicien vacíos tiempo_seleccionado.set("ninguno")</pre>
¿Cómo usó o adaptó la respuesta?	Se usó en varias partes del programa, especialmente en la configuración, ya que esa parte usa bastante de radios. No cambié el código y sirvió
Reflexión crítica	Si sirvió bastante, aclaro las dudas que tenía.

Capturar el click en la equis de una ventana

Se hizo uso de la IA saber capturar el click en la equis de una ventana.

Objetivo del uso	Obtener el estado de un widget cualquiera con <code>cget("state")</code>
Herramienta utilizada	Gemini
Prompt o pregunta	¿Cómo puedo capturar el click en la equis de una ventana?
Respuesta	<code>root_config.destroy()</code> <code>root_config.protocol("WM_DELETE_WINDOW", guardar_y_cerrar)</code>
¿Cómo usó o adaptó la respuesta?	La IA me dió dos opciones, cuando se cierra la ventana, y otra para llamar una función cuando esto suceda
Reflexión crítica	Use ambas en algunas partes, más que todo para cuando el usuario cierra la configuración, que se guarden los datos que puso.

Ayuda con formatear segundos

Se hizo uso de la IA para mejorar una función de convertir segundos.

Objetivo del uso	Pointer formato de 00 a horas minutos y segundos
Herramienta utilizada	Gemini
Prompt o pregunta	¿Cómo puedo establecer que los segundos horas o minutos sean 00 si no hay?
Respuesta	# El :02d le dice a Python: "Muestra este número con 2 dígitos, rellenando con ceros a la izquierda si falta" <code>formato = f"{horas:02d}:{minutos:02d}:{segundos:02d}"</code>
¿Cómo usó o adaptó la respuesta?	La use así como está, no le cambie mucho, solamente el nombre

Reflexión crítica	Me sirvió bastante, porque no quería que se viera feo en el informe de topx
-------------------	---

Algoritmo de ordenamiento

Se hizo uso de la IA para ordenar los mejores tiempos.

Objetivo del uso	Algoritmo para ordenar los tiempos
Herramienta utilizada	Gemini
Prompt o pregunta	Tengo un diccionario con varios niveles de dificultad, donde cada nivel tiene partidas con tiempos establecidos, quiero que se ordenen del menor al mayor
Respuesta	<pre> for i in range(n): for j in range(0, n - i - 1): if lista[j]["tiempo"] > lista[j + 1]["tiempo"]: # Intercambio de posiciones temporal = lista[j] lista[j] = lista[j + 1] lista[j + 1] = temporal </pre>
¿Cómo usó o adaptó la respuesta?	Me sirvió bastante para ordenar los tiempos de menor a mayor, no lo cambie, lo use así como está
Reflexión crítica	Sirvió bastante, no produjo ningún error.

Error en pausar reloj

Se hizo uso de la IA para arreglar un error en el reloj.

Objetivo del uso	Le pedí ayuda porque tenía un error cuando intentaba parar el cronómetro en modo de cronómetro con tiempo extra, después de usar el timer.
Herramienta utilizada	Gemini

Prompt o pregunta	¿Cómo calculo el tiempo inicial del timer si se está usando tiempo extra de este?
Respuesta	<code>root_juego.hora_inicio = datetime.now() - timedelta(seconds=root_juego.remaining_time)</code>
¿Cómo usó o adaptó la respuesta?	Lo puse así como esta, resolvió el problema, no cambie nada
Reflexión crítica	Sirvió bastante, no produjo ningún error.

Obtener el atributo de un widget

Se hizo uso de la IA para entender para qué sirve `getattr`.

Objetivo del uso	Le pedí ayuda que me explicara para qué sirve <code>getattr()</code>
Herramienta utilizada	Gemini
Prompt o pregunta	¿Para qué sirve <code>getattr</code> ?
Respuesta	<code>getattr()</code> es una función nativa de Python súper útil y poderosa que sirve para buscar y obtener el valor de una variable o función dentro de un objeto usando su nombre escrito como texto (un string). <code>getattr(objeto, "nombre_del_atributo", valor_por_defecto)</code>
¿Cómo usó o adaptó la respuesta?	Lo use en el cronómetro para traer el valor de juego iniciado y compararlo en un <code>if</code> , además del juego pausado
Reflexión crítica	Me sirvió mucho, no lo cambié, lo deje como esta

Obtener `datetime` actual, formato estándar

Se hizo uso de la IA para obtener una fecha actual, con el formato estándar dado por el profesor, ISO 8601.

Objetivo del uso	Use la IA para que me ayude a obtener una fecha actual, en el formato de ISO 8601
Herramienta utilizada	Gemini
Prompt o pregunta	Ocupo obtener una fecha y hora estándar ISO 8601, formato básico AAAMMDDTHHMMSS. Usando python
Respuesta	<pre>from datetime import datetime, timezone ahora_utc = datetime.now(timezone.utc) fecha_iso_utc = ahora_utc.strftime("%Y%m%dT%H%M%SZ")</pre>
¿Cómo usó o adaptó la respuesta?	Lo implemente para guardar la fecha de la bitácora de jugadas, lo use así y me sirvió
Reflexión crítica	Me ayudó mucho a simplificar el trabajo de buscar, y no me dió errores.

Estructura de los archivos usados

Esta sección de la documentación es para especificar la estructura de cada tipo de archivo .JSON que se usó en el programa, además de el tipo de algoritmo random usado para el número de nivel del juego.

Algoritmo de selección aleatoria

El programa posee una serie de partidas que previamente han sido registradas y de ahí selecciona aleatoriamente una partida según el nivel de dificultad configurado. Este algoritmo de selección debe usarse para seleccionar alguna de las partidas de tal forma que siempre se elija una al azar.

El algoritmo usado es mediante el módulo de python conocido como random. Este módulo debe importarse al principio del programa y fue implementado de la siguiente manera.

Cabe destacar, se está usando una condición, ya que existe una funcionalidad que permite borrar el juego y volver a desplegar la ventana de jugar, sin embargo, esta debe tener el mismo número de partida que tenía previamente, por ello, está la condición, para poner uno al azar si no se especificó que se ha borrado el juego.

```
if juego_borrado_numero_partida == " ":
```

```
    #Se genera un número aleatorio del 1 al 3 incluyendolos
```

```
    root_juego.numero_partida = str(random.randint(1, 3))
```

```
else:
```

```
    root_juego.numero_partida = juego_borrado_numero_partida
```

Archivo "sudoku2026juegoactual"

Este archivo guarda el juego actual (cuadrícula) con su configuración (nivel, uso del reloj, etc.) y nombre del jugador. El objetivo es que el jugador pueda en cualquier momento guardar el juego y posteriormente continuarlo.

Para la creación de este archivo se usó un tipo json, ya que se debía guardar información y es el que más se sabía usar hasta el momento.

Su estructura es la siguiente. Al iniciar el programa este archivo se encuentra en blanco, por lo cual se le inserta un diccionario vacío, para ir agregando en este las partidas que se irán jugando y guardando.

Para agregar una partida se debe usar como llave en el diccionario el nombre del jugador. Luego, cada jugador tiene asociado un diccionario, que contiene como llave la dificultad (fácil, intermedio, difícil).

Cada dificultad tiene asociado otro diccionario, que tiene por llaves a la cuadrícula y al reloj. La cuadrícula tiene un diccionario que posee por llaves las posiciones de cada elemento y su valor asociado.

En cambio, el reloj tiene una lista, que posee el tipo de reloj que se está usando, y el valor asociado de las horas, los minutos y los segundos.

```
{ "sa": {
  "f\u00e1cil": {
    "cuadr\u00edcula": {
      "0,0": 3, "0,3": 2, "0,8": 1, "1,4": 4, "2,1": 5, "2,5": 8, "3,2": 9,
      "3,7": 4, "4,0": 8, "4,4": 1, "4,8": 2, "5,1": 6, "5,6": 7, "6,3": 5,
      "6,7": 1, "7,4": 9, "7,7": 7, "8,0": 7, "8,5": 6, "8,8": 3
    },
    "reloj": ["cronometro", "00", "00", "10"]
  }
}
```

Archivo "sudoku2026configuración"

Esté archivo indica las condiciones con que se va a jugar. Contiene los siguientes datos.

Para la creación de esté archivo se usó un tipo json.

Esté archivo se crea por defecto al iniciar el juego, ya que debe haber una configuración predeterminada por si el usuario desea empezar a jugar una vez entra al programa. Su estructura es la siguiente.

El archivo contiene un diccionario, donde sus llaves son el "nivel" (dificultad) del juego, el tipo de "reloj" que se está usando, el "top x", y los "elementos".

El nivel tiene por valor la dificultad. El reloj tiene una lista con el tipo de reloj y los tiempos iniciales (pueden ser diferentes de cero si el usuario eligió un timer). El top x tiene 0 por defecto, pero el usuario puede modificarlo. Y los elementos pueden tener las letras o los números.

```
{ "nivel": "fácil",  
  "reloj": ["cronometro", 0, 0, 0],  
  "top x": 0, "elementos": "numeros"  
}
```


Conclusiones

En este apartado se detallan las conclusiones relacionadas con la creación del programa, entre estos: problemas encontrados y el aprendizaje obtenido.

Problemas Encontrados

Se enumera a continuación, cada problema que se encontró llevando a cabo el proyecto, así como sus soluciones implementadas.

1. Aprender desde cero a usar Tkinter. Nunca había escuchado sobre la biblioteca de Tkinter, no sabía que python tenía una herramienta para crear inserta vez gráficas. Mi mayor problema fue investigar, no sabía nada, debía empezar desde cero.

Solución: me puse a ver muchos videos explicativos, busqué información en todas las páginas que pude, y pregunté bastantes cosas a la IA.

2. A la hora de programar. Encontré un error con los radio buttons, no podía poner uno por default, es decir, que se marcará al menos uno, aunque el usuario no seleccione nada

Solución: le pregunté a la IA como podía hacer para marcar uno por determinado, y me explicó como hacerlo y el por qué debía hacerlo.

3. Igualmente con los radio buttons, cuando intente meterlos en la ventana secundaria todos se seleccionan al mismo tiempo, y tenían comportamientos extraños.

Solución: investigando y viendo documentación de otras páginas, me di cuenta que debía poner el nombre de la ventana secundario en vez de la principal, a la hora de definir los elementos.

4. Tenía un problema cuando requería usar el reloj, ya que el timer que usa tiempo extra, no estaba funcionando como debía.

Solución: hice uso de la IA, me ayudó a obtener el tiempo y a usar un pedazo de código para que sirviera.

5. Surgió un problema con el cronómetro que tenía, ya que me basé en uno que encontré en internet, lo malo es que ocupaba un formato que rellena con ceros a la izquierda de ser necesario.

Solución: usando la IA me ayudó a formar este formato e implementarlo en el proyecto.

Aprendizajes Obtenidos

- Importancia de los tipos de archivos.
- Uso intensivo del tipo de archivo .JSON así como funciones de lectura y escritura.
- Uso de programación dirigida por eventos, así como su importancia.
- Desarrollar una interfaz gráfica usando la librería de tkinter, así como sus elementos, widgets, tablas, módulos.
- La importancia de una interfaz gráfica en los programas de desarrollo.
- Uso de datetime para implementar un cronómetro y definir una fecha según el estándar ISO 8601
- Uso de pilas mediante los métodos de push() y pop().
- Capturar el click en la equis de una ventana.

¿Qué haría diferente en un próximo proyecto?

1. Empezaría la documentación interna desde que comienzo a escribir código y plantear soluciones

2. Hubiera dividido la investigación de los widgets de Tkinter, porque muchas veces veía a profundidad algunos como el button y otros los dejaba por ahí, después los necesitaba y debía ir otra vez a buscar más información, entonces me atrasé mucho.
3. Haría un documento aparte de esté para ir tachando lo que voy haciendo y lo que hace falta, porque a veces me faltaban cosas porque me costaba saber que eran y ver donde estaban las instrucciones.
4. Otra vez dejaría los detalles para el final. Sigo gastando mucho tiempo en partes del programa que no son funcionalidades, solo quiero que se vean bien, porque gasto mucho tiempo tratando de mejorarlas.
5. No me confiaría, al principio iba muy bien, pero como se alargo la entrega descuide el programa, y todavía lo estoy terminando el mismo día de la entrega.

Estadística de tiempos

A continuación una tabla con la estadística de tiempo, en horas invertidas, con respecto a la elaboración de cada aspecto del proyecto. La estadística permite medir el esfuerzo dedicado al trabajo en términos de actividades y tiempos.

Actividad Realizada	Horas
Análisis del problema	5
Diseño de algoritmos	3
Investigación	7
Programación	36
Documentación interna	2
Pruebas	2
Elaboración del manual de usuario	2
Elaboración de documentación del proyecto	8
TOTAL	65

Lista de Revisión del Proyecto

Concepto	Puntos originales	Avance 100/%/0	Puntos obtenidos	Análisis de resultados
Menú	1	100%	1	
Opción Jugar (despliegue del juego)	15	100%	15	
Botón Iniciar Juego	10	100%	10	
Crear Top X	5	100%	5	
Botón de Deshacer Jugada	8	100%	8	
Botón Rehacer Jugada	8	100%	8	
Botón Borrar Juego	3	100%	3	
Botón de Terminar Juego	3	100%	3	
Botón Top X	6	100%	6	
Formato PDF	2	100%	2	
Botón Guardar Juego	8	100%	8	
Botón Cargar Juego incluyendo su despliegue	8	100%	8	
Opción Configurar	8	100%	8	
Ayuda (manual de usuario)	5	100%	5	
Acerca de	1	100%	1	
Salir	1	100%	1	
Reloj: cronómetro y timer tiempo real	8	100%	8	
TOTAL	100	100%	100	

Referencias bibliográficas

<https://www.adobe.com/es/acrobat/resources/document-files/data-files.html>

<https://mpru.github.io/introprog/uso-de-archivos-de-datos.html>

<https://www.arsys.es/blog/archivo-json-que-es-y-para-que-sirve>

<https://www.adobe.com/acrobat/resources/document-files/what-is-a-csv-file.html>

<https://www.ionos.com/es-us/digitalguide/servidores/configuracion/archivo-txt/>

<https://www.techtarget.com/whatis/definition/binary-file>

<https://profile.es/blog/programacion-orientada-a-eventos/>

https://www.lenovo.com/es/es/glossary/event-driven-programming/?orgRef=https%253A%252F%252Fwww.google.com%252F&srsId=AfmBOopB65GuG6d_97k2Uqz7y0GgkDJpT67dFkl4JN2aQjSZxyF9-QYO

<https://www.lenovo.com/mx/es/glosario/que-es-una-interfaz-grafica-de-usuario/?orgRef=https%253A%252F%252Fwww.google.com%252F&srsId=AfmBOor4SeTrxkME8Amgo3ttRPaZ38RiMQYGCctXKtJlw3aNBOEiqXhD>

<https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/que-es-una-gui/>

<https://www.pythonguis.com/tutorials/create-gui-tkinter/>

<https://realpython.com/python-gui-tkinter/>

<https://www.pythonguis.com/tutorials/tkinter-basic-widgets/>

<https://docs.python.org/es/3.8/library/tkinter.ttk.html>

<https://www.pythontutorial.net/tkinter/tkinter-radio-button/>

<https://www.activestate.com/resources/quick-reads/how-to-display-data-in-a-table-using-tkinter/>

<https://recursospython.com/guias-y-manuales/ventanas-secundarias-tkinter/>

<https://recursospython.com/guias-y-manuales/cuadros-de-dialogo-messagebox-en-tkinter/>

<https://python-course.eu/tkinter/events-and-binds-in-tkinter.php>

<https://parzibyte.me/blog/posts/cronometro-tkinter-python/>

<https://dev.to/ashfin-ahmed-kp/creating-a-countdown-timer-with-tkinter-in-python-4i3j>

<https://medium.com/@monalisha1/building-a-simple-countdown-timer-application-using-python-tkinter-aa2fa9476982>

<https://recursospython.com/guias-y-manuales/iconos-en-ventanas-de-tk-tkinter/>

<https://www.geeksforgeeks.org/python/how-to-use-images-as-backgrounds-in-tkinter/>

<https://www.tutorialspoint.com/article/how-do-i-change-button-size-in-python-tkinter>

<https://www.pythontutorial.net/tkinter/tkinter-event-binding/>

<https://www.pythontutorial.net/tkinter/tkinter-command/>

<https://tkdocs.com/tutorial/eventloop.html>

<https://www.pythonguis.com/tutorials/create-ui-with-tkinter-grid-layout-manager/>

<https://www.pythontutorial.net/tkinter/tkinter-entry/>

<https://docs.python.org/es/3.13/library/tkinter.ttk.html>

<https://www.pythontutorial.net/tkinter/tkinter-button/>

<https://recursospython.com/guias-y-manuales/boton-button-en-tkinter/>

<https://enriquelazcorreta.gitbooks.io/tkinter/content/introduccion-a-la-programacion-visual/la-programacion-orientada-a-eventos.html>

<https://www.tutorialspoint.com/article/placeholder-in-tkinter>

<https://www.tutorialspoint.com/article/how-to-create-a-child-window-and-communicate-with-parents-in-tkinter>